

Enhancing LLMs for Graph Tasks via Graph-aware LoRA Generation

Junshu Sun^{1,2} Wanxing Chang³ Qingming Huang² Shuhui Wang¹

Abstract

Graph neural networks (GNNs) tightly couple their input-output parameters to dataset-specific feature spaces and target sets, exhibiting limited transferability across different datasets. In contrast, language models (LMs) generalize flexibly via a unified input-output interface, motivating recent attempts to adapt LMs to graph tasks. However, existing methods struggle to encode whole-graph information, leading to potential information loss and suboptimal graph understanding. In this work, we propose a novel weight-level information injection paradigm for adapting LMs to graph tasks. This paradigm injects whole-graph information by generating task-specific weight updates that interact directly with hidden representations. Instantiating this paradigm following low-rank adaptation (LoRA), we introduce GaRA, a Graph-aware LoRA generation model. GaRA constructs low-rank weight updates conditioned on the original graph structures and constrains the norm of the generated updates, thus injecting whole-graph information and avoiding the optimization bias in the weight generation. Empirical studies demonstrate that GaRA consistently outperforms baselines on zero-shot graph learning tasks.

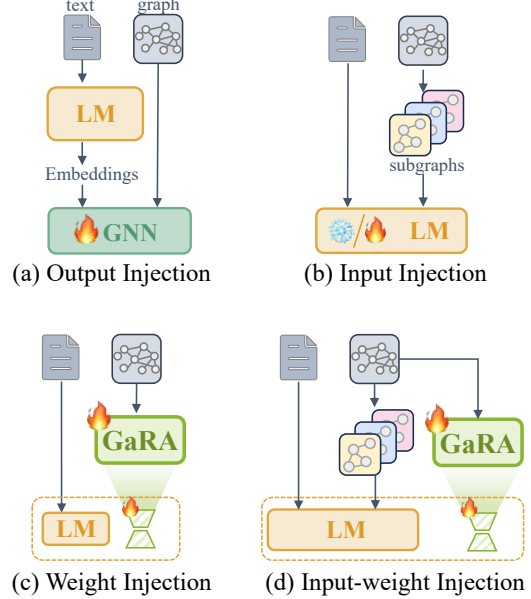


Figure 1. Comparison between Graph Injection Paradigms.

(a) Output-level injection paradigm employs GNNs as the model predictor, failing to tackle graphs with diverse labels. (b) Input-level injection paradigm transforms input graphs into subgraph sequences, overlooking higher-level information in the original graphs and leading to potential information loss. (c) Our weight-level injection paradigm enables LMs to learn from whole-graph information while maintaining the ability to tackle diverse tasks. (d) In practice, weight-level injection is complementary to the input-level injection, alleviating potential information loss.

1. Introduction

Graph neural networks (GNNs) have emerged as effective models for supervised graph representation learning, addressing tasks such as node classification (Kipf & Welling, 2017; Sun et al., 2025), link prediction (Zhang & Chen, 2018), and graph classification (Ying et al., 2021; Sun et al.,

2023) across diverse application scenarios (Li et al., 2021; Zhang et al., 2022; Bessadok et al., 2023). However, the parameterization of GNNs is inherently tied to the number of input feature dimensions and output targets (e.g., labels or attributes) in the training data (Zhao et al., 2024). As graphs from different datasets exhibit heterogeneous feature spaces or target sets (Chen et al., 2024b), pre-trained GNNs cannot be directly transferred to downstream graphs without modifying model parameters or architectures (Zhao et al., 2024). In contrast, language models (LMs) operate on a unified textual interface, taking textual sequences as both inputs and outputs (Chen et al., 2024a; Li et al., 2024a). By varying the length and content of input sequences, LMs naturally accommodate heterogeneous feature descriptions and produce predictions via textual generalization, enabling

¹State Key Lab. of AI Safety, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China ²University of Chinese Academy of Sciences, Beijing, China ³DAMO Academy, Alibaba Group, Hangzhou, China. Correspondence to: Shuhui Wang <wangshuhui@ict.ac.cn>.

flexible transfer across tasks and domains.

Motivated by the flexibility of LMs, recent studies have explored their application to graph learning (Chen et al., 2024a). A major challenge in this line of work is enabling the model to effectively capture graph topology (Chen et al., 2024a). Mainstream solutions can be broadly categorized into *output-level* and *input-level* injecting methods. Output-level injection leverages GNNs to refine LM representations and produce final predictions (Liu et al., 2023). However, similar to traditional GNNs, these methods remain tightly coupled to task-specific target sets and thus exhibit limited transferability across different downstream tasks. Differently, input-level injection incorporates structural information directly into LM inputs by transforming graph structures into prompt sequences, and subsequently fine-tunes pre-trained LMs to adapt to these prompts (Chen et al., 2024a; Wang et al., 2025). Nevertheless, due to the fully connected self-attention mechanism, LMs incur $O(n^2)$ computational complexity for an input sequence of length n . As a result, graph-based prompts are typically restricted to small subgraphs, discarding the remaining structural context. This limitation overlooks higher-level information encoded in the original graph and potentially leads to information loss and biased graph understanding.

To alleviate information loss, direct solutions include expanding the subgraph size or concatenating the global graph feature to the input sequence, yet increasing the computational overhead quadratically. This suggests that simply augmenting information volume at the input level is not a viable solution. In this paper, we propose a novel *weight-level* injection paradigm for adapting LMs to graph tasks. Specifically, the injection is realized by generating the weight matrix with the whole-graph information and applying the resulting weights to the representations in the hidden layers. Our weight-level injection complements the subgraph information injected at the input level, alleviating potential information loss while avoiding a quadratic increase in computational cost.

We then implement this weight-level paradigm with low-rank adaptation (LoRA), proposing the **Graph-aware LoRA** generation method, named GaRA, including a LoRA-style weight generation module and a norm-rescaling module. The weight generation module generates a low-rank matrix to construct weight updates, injecting whole-graph information into the LM backbone. The norm-rescaling module is proposed to tackle the direction-preserving norm amplification problem, which we prove as an architectural optimization bias during fine-tuning. In the empirical analysis, we show that GaRA can consistently surpass baseline methods on zero-shot transfer tasks. Our contributions can be summarized as follows:

- We design a novel weight-level information-injecting

paradigm, which preserves the whole-graph information for LMs on graph tasks.

- We propose a graph-aware weight generation method, GaRA, under the weight-level paradigm, which alleviates potential information loss and tackles the architectural optimization bias problem.
- We demonstrate the advantages of GaRA on zero-shot graph learning tasks, which surpass baselines on different transfer tasks.

2. Related Work

LMs have demonstrated predominant generalizability across a wide range of tasks (OpenAI et al., 2024). In light of this, substantial efforts have been devoted to adapting LMs to graph tasks (Chen et al., 2024a). However, Wang et al., (2023) suggest that pure LMs struggle to capture structural relations in graphs. To address this limitation, recent studies incorporate graph structures into LMs (Chen et al., 2024a), enabling more effective adaptation to graph tasks. Existing work can be broadly categorized into output-level and input-level injecting methods.

2.1. Output-level Injection

In these methods, LMs typically serve as text encoders, while GNNs further enhance the representations and make final predictions (Liu et al., 2023; Chen et al., 2024b). Owing to the strong textual understanding capabilities, LMs facilitate semantic alignment across diverse real-world graphs, thereby reducing domain gaps in applications such as biochemical graphs (Morris et al., 2020), social networks (Lim et al., 2021), and e-commerce graphs (Hu et al., 2020). Given the LM-encoded textural features, GNNs further inject structural features through message passing, ensuring graph-awareness of the final predictions. Nevertheless, the GNN-based predictors fail to tackle graphs with diverse labels (Zhao et al., 2024), leading to limited adaptability of the output-level injection. Due to this limitation, we follow the input-level injecting methods to construct our framework in this paper.

2.2. Input-level Injection

In contrast to output-level methods, input-level injection adopts LMs as predictors. Structural information is directly injected into the LM inputs by concatenating graph prompts, thereby enabling graph-aware learning. Typically, the graph prompts can be derived from GNN outputs (Wang et al., 2024; Zhang et al., 2024; Lv et al., 2025; He et al., 2025) or graph compression strategies (Chen et al., 2024a; Wang et al., 2025).

GNN-based Prompting. GNN-based prompting strategy encodes graph structures into prompt embeddings with various GNN models (Kipf & Welling, 2017; Hamilton et al., 2017; Sun et al., 2024). However, to ensure computational efficiency and training stability, these methods commonly freeze the GNN during LM tuning (Wang et al., 2024; Li et al., 2024a). In consequence, the resulting graph-dependent signals cannot be optimized for downstream objectives, which may lead to suboptimal representations and exacerbate distribution shifts between pre-training and downstream data (Wang et al., 2025). To address this problem, GOFA (Kong et al., 2024) interleaves GNN layers into a frozen pre-trained LLM, adjusting the prompt signal accordingly but also incurring intensive computational requirement.

Graph Compression Prompting. Unlike GNN-based prompting, graph compression strategies leverage the relational modeling capacity of pre-trained LMs, such as large language models (LLMs), to learn prompts from node sequences. A key challenge in this paradigm is transforming input graphs into sequences while preserving structural information. To this end, LLaGA (Chen et al., 2024a) and ZeroG (Li et al., 2024a) generate node sequences via neighborhood sampling for each node¹. However, the sampling strategy is permutation-variant, making the compressed representations sensitive to the ordering of the sampled nodes. In contrast, UniGTE (2025) restores permutation invariance by fixing relative positional encodings between nodes to zero and injecting graph structure via attention biases.

Beyond the characteristics discussed above, due to constraints on model complexity and the goal of task unification, both GNN-based methods and graph compression strategies reduce all tasks to the graph level by extracting an n -hop subgraph centered on the target node or edge. This process overlooks higher-level information encoded in the original graph, potentially leading to information loss in downstream tasks. Different from these methods that solely operate at the input level, we propose a weight-level paradigm, injecting whole-graph information via weight generation.

2.3. Weight-level Injection

Weight-level injection has emerged as a promising paradigm for adapting LMs to the visual data. For example, VLoRA (Ma et al., 2024) extracts informative features via learnable queries and transforms them into low-rank weight updates. Adapting this paradigm to graph tasks involves two challenges. First, graph datasets typically contain much less training data than vision datasets, making weight generation

¹ZeroG also applies message passing after the LM. Since this process introduces no learnable parameters, we categorize ZeroG as an input-level injecting method.

more prone to optimization bias during fine-tuning. Second, graph data encodes rich relational structures and multi-scale information, making it challenging to inject sufficient features via a small number of generated weight updates. To address these challenges, GaRA employs norm rescaling in weight injection to avoid optimization bias. The global features captured by weight injection are combined with the local features from input injection, thereby enabling more sufficient feature injection.

3. Preliminaries

3.1. Notations

Given an input graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, \dots, v_n\}$ denotes the set of nodes with $|\mathcal{V}| = n$, and $\mathcal{E} = \{e_{i,j} \mid v_j \in \mathcal{N}(v_i)\}$ denotes the set of edges with $|\mathcal{E}| = m$. For a node $v \in \mathcal{V}$, let $\mathcal{N}(v)$ denote its one-hop neighborhood. Each node v is associated with a feature vector $\mathbf{x}_v \in \mathbb{R}^{d_{\text{in}}}$ and a textual description t_v , where d_{in} denotes the feature dimension. Stacking all node features yields the node feature matrix $\mathbf{X} = (\mathbf{x}_{v_1}, \dots, \mathbf{x}_{v_n})^\top \in \mathbb{R}^{n \times d_{\text{in}}}$. Traditional GNNs are sensitive to the specific feature dimension d_{in} of different graphs. To tackle the heterogeneous feature spaces, pre-trained LMs (Devlin et al., 2019; Reimers & Gurevych, 2019) are adopted to embed textual descriptions into $\mathbf{X}$, unifying input features in the embedding space. Graph structures can be represented by the adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, where $\mathbf{A}_{i,j} = 1$ if $v_j \in \mathcal{N}(v_i)$ and 0 otherwise.

3.2. Model Fine-tuning

During the adaptation of the LMs, fine-tuning strategies are adopted for pre-trained models. Without loss of generality, given a weight matrix $\hat{\mathbf{W}} \in \mathbb{R}^{p \times q}$ for a linear transformation $\mathbb{R}^p \mapsto \mathbb{R}^q$, the general adaptation process can be formulated as an additive modification to the original weights

$$\text{(General)} \quad \mathbf{W} = \hat{\mathbf{W}} + \Delta\mathbf{W}, \quad (1)$$

where $\Delta\mathbf{W}$ denotes the weight update. Among different strategies, low-rank adaptation (LoRA) achieves pronounced efficiency and tuning effectiveness (Hu et al., 2021). Given a weight matrix $\hat{\mathbf{W}} \in \mathbb{R}^{p \times q}$, LoRA introduces two learnable low-rank matrices $\mathbf{L} \in \mathbb{R}^{p \times r}$, $\mathbf{R} \in \mathbb{R}^{q \times r}$ to enable parameter-efficient fine-tuning. The weight update can be reparameterized as

$$\text{(LoRA)} \quad \mathbf{W} = \hat{\mathbf{W}} + \Delta\mathbf{W} = \hat{\mathbf{W}} + \mathbf{L}\mathbf{R}^\top \quad (2)$$

When $r \ll \min(p, q)$, LoRA significantly reduces memory consumption and improves training efficiency by optimizing only the low-rank matrices while keeping $\hat{\mathbf{W}}$ fixed.

4. Weight-level Graph Injection

Despite improved efficiency and enhanced adaptability, input-level injection strategies discard higher-level information in the whole-graph context, potentially resulting in information loss and suboptimal adaptation of LMs to graph tasks. To alleviate information loss, we propose a novel weight-level paradigm, injecting whole-graph information via weight generation and complementary to the input-level subgraph information.

4.1. Task Description

To achieve task unification, both node and edge tasks are reformulated by extracting task-specific subgraphs as graph-level inputs (Tang et al., 2024). Each task is then represented by a tuple $(\mathcal{G}, \mathcal{G}_s, t_{\text{task}}, t_{\text{label}})$, where $\mathcal{G}_s \subseteq \mathcal{G}$ is the subgraph associated with the task, t_{task} is a textual task description specifying the prediction objective, and t_{label} is the corresponding textual label. Graph-level tasks are treated as a special case where $\mathcal{G}_s = \mathcal{G}$. Under this formulation, an LM generates the label description $\hat{t}_{\text{label}}$ conditioned on $\mathcal{G}$, $\mathcal{G}_s$, and t_{task} .

Motivated by the ability to bridge the gap between pre-training and downstream data, we adopt the graph compression strategy for our injection framework. It fine-tunes an LLM encoder that compresses the input sequence $t_{\text{enc}} = [t_{\mathcal{G}_s} \| t_{\text{task}}]$ into a continuous graph prompt, where $\|$ denotes concatenation, $t_{\mathcal{G}_s} = \{t_v | v \in \mathcal{G}_s\}$ denotes the subgraph node sequence. A frozen LLM decoder then takes the learned prompt and the task description t_{task} as input for label generation.

4.2. Injecting Graph Structures via Weight Generation

General Formulation. Given the input feature matrix $\mathbf{X}$ and the adjacency matrix $\mathbf{A}$, we employ a GNN model to capture graph information in the whole-graph context

$$\mathbf{h} = \text{MaxPool}(\text{GNN}(\mathbf{X}, \mathbf{A})), \quad (3)$$

where $\mathbf{h} \in \mathbb{R}^d$ denotes the resulting graph-level representation. In Eq. 3, the GNN learns node-level features via message passing on the input graph, encoding the graph topology implicitly. The subsequent MaxPool operation lifts node features into a graph-level representation, providing whole-graph information for graph-aware weight generation.

Based on the graph-level representation $\mathbf{h}$, a non-linear mapping $\mathfrak{f} : \mathbb{R}^d \rightarrow \mathbb{R}^{p \times q}$ is adopted to generate the weight update, reformulating Eq. 1 as

$$\mathbf{W} = \hat{\mathbf{W}} + \Delta\mathbf{W} = \hat{\mathbf{W}} + \mathfrak{f}(\mathbf{h}; \Theta), \quad (4)$$

where Θ denotes the parameter matrix. To ensure the generated matrix covering both positive and negative values

symmetrically, $\tanh$ is employed in $\mathfrak{f}$.

Optimization Bias. Graph-aware weight generation injects whole-graph information into the adaptation process without incurring a quadratic increase in computational cost. However, the primary formulation in Eq. 4 introduces an architectural bias during fine-tuning, under which the weight update is prone to norm amplification. Consequently, when increasing the norm of $\Delta\mathbf{W}$ decreases the training loss, gradient descent will try to amplify $\|\Delta\mathbf{W}\|$. This results in an architectural optimization bias toward norm-scaling weight update.

Formally, we show that for an adaptation strategy defined in Eq. 4, there always exists a parameter update such that $\Delta\mathbf{W}$ can be scaled with its direction unchanged.

Theorem 4.1 (Direction-preserving Update). *Let $\Delta\mathbf{W} = \mathfrak{f}(\mathbf{h}; \Theta)$. For a fixed input $\mathbf{h}$ and the Jacobian $J_{\mathfrak{f}}(\Theta) = \partial \mathfrak{f}(\mathbf{h}; \Theta) / \partial \Theta$, there exists a nonzero parameter direction $\delta\Theta$ such that under the update $\Theta \leftarrow \Theta + \epsilon \delta\Theta$, the weight update satisfies $\Delta\mathbf{W} \leftarrow (1 + \epsilon) \Delta\mathbf{W} + O(\epsilon)$.*

Proof is provided in the Appendix A. Compared to the weight generation, the scaling of $\Delta\mathbf{W}$ in traditional fine-tuning methods arises solely from the scaling of the parameter matrices, which are discouraged by regularization techniques such as weight decay. However, when $\Delta\mathbf{W}$ is generated by the parameteric function $\mathfrak{f}(\mathbf{h}; \Theta)$, weight decay applied to Θ does not directly constrain the norm of $\Delta\mathbf{W}$. Therefore, the direction-preserving norm amplification still emerges even under regularization.

4.3. GaRA: Graph-aware LoRA Generation

LoRA-style Generation. Motivated by the efficiency and effectiveness of LoRA for model fine-tuning, we employ the low-rank strategy in our weight generation process (Fig. 2). Specifically, take the left matrix generation as an example, the weight update in Eq. 2 can now be reformulated as

$$\mathbf{W} = \hat{\mathbf{W}} + (\mathbf{L} + \mathfrak{g}(\mathbf{h}; \Theta))\mathbf{R}^\top, \quad (5)$$

where $\mathfrak{g} : \mathbb{R}^d \rightarrow \mathbb{R}^{d_M \times r}$ generates the low-rank matrices $\mathbf{G} = \mathfrak{g}(\mathbf{h}; \Theta)$ for LoRA, $d_M = p$ when $\mathbf{G}$ serves as an augmented left low-rank matrix, and $d_M = q$ when it serves as an augmented right matrix. In practice, we combine linear mapping with $\tanh$ to implement $\mathfrak{g}$. During fine-tuning, both Θ and the low-rank matrices $\mathbf{L}/\mathbf{R}$ are trainable. Empirically, augmenting the left low-rank matrix generally achieves better adaptation results. Please refer to Sec. 5.3.3 for more details. Therefore, in the following context, we stick to left matrix generation to formulate specific designs in GaRA.

The direction-preserving norm amplification problem also exists in the LoRA framework, as the results in Theorem 4.1

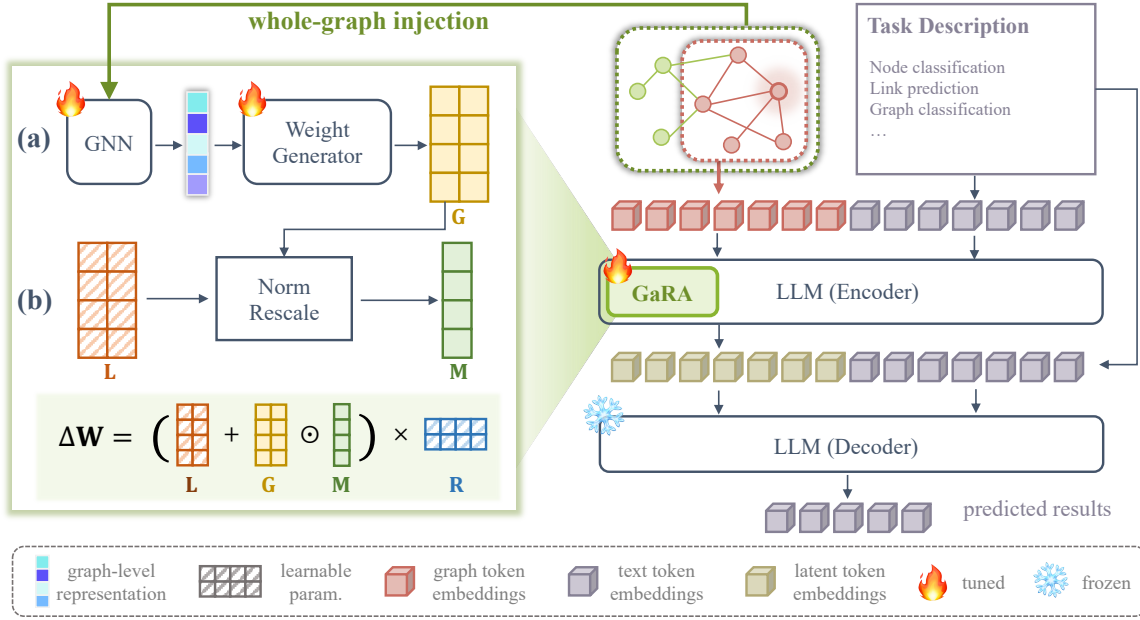


Figure 2. **Pipeline of the Graph-aware LoRA Generation model, GaRA.** (a) To inject whole-graph information into LLMs, GaRA first applies GNN to the whole input graph, extracting a global representation via max pooling. This representation is then fed into an MLP-based weight generator to obtain low-rank weight updates $\mathbf{G}$. (b) To tackle the norm-amplification optimization bias, the norm of $\mathbf{G}$ is further rescaled based on the left low-rank matrix $\mathbf{L}$ from LoRA. The norm-rescaled $\mathbf{G}$ is combined with the low-rank matrices in LoRA to construct the final weight updates.

directly extend to the low-rank setting. Proof is presented in the Appendix A.

Corollary 4.2 (Direction-preserving Update in LoRA-style Generation). *Let $\Delta\mathbf{W} = \mathbf{L}\mathbf{R}^\top + g(\mathbf{h}; \Theta)\mathbf{R}^\top$. For a fixed input $\mathbf{h}$ and the Jacobian $J_g(\Theta) = \partial g(\mathbf{h}; \Theta)/\partial \Theta$, there exists a nonzero parameter direction $\delta\Theta$ such that under the update $\Theta \leftarrow \Theta + \epsilon\delta\Theta$, the weight update satisfies $\Delta\mathbf{W} \leftarrow (1 + \epsilon)\Delta\mathbf{W} + O(\epsilon)$.*

These theoretical results can be verified in the empirical evaluation. During the training process, we observe that while LoRA exhibits a stable training dynamics, the primary LoRA-style generation method leads to continuous norm amplification on the generated low-rank matrix. Please refer to Sec. 5.2 for more details.

Influence of Norm Amplification. Norm amplification creates an optimization shortcut during fine-tuning, which potentially leads to overfitting. Specifically, traditional LoRA uses weight decay to regularize the norm of weight updates, yielding much smaller values than the original weight. As a result, minimizing the loss typically requires optimizing both the update and the original weight. In contrast, the existence of direction-preserving updates in weight generation scales the norm of weight updates, weakening the contribution of the original weight. Therefore, the optimizer can solely rely on the generated weight updates to reduce the training loss, making it easier to fit the fine-tuning

dataset and thus preferred during optimization. Such an optimization bias gradually dilutes the knowledge encoded in the original weight via large weight updates, leading to ability degradation for general tasks and model overfitting. This motivates us to introduce additional constraints in preventing direction-preserving norm amplification.

Constraining Norm Amplification. To tackle the unconstrained norm amplification bias during optimization, we propose a simple yet effective solution by fixing the average column norm of the generated low-rank matrix $\mathbf{G}$ to match that of the corresponding left matrix $\mathbf{L}$. This choice is motivated by the fact that both $\mathbf{L}$ and $\mathbf{G}$ serve as the left factor to construct the weight update in the same manner. As a result, the norm of $\mathbf{L}$ provides a natural and stable reference scale without introducing additional hyperparameters. With the norm rescaling, Eq. 5 can be reformulated as

$$\begin{aligned} \mathbf{W} &= \hat{\mathbf{W}} + (\mathbf{L} + \mathbf{G} \odot \mathbf{M})\mathbf{R}^\top, \\ \text{(GaRA)} \quad \mathbf{M} &= \frac{\text{avg}_j(\text{norm}(\mathbf{L}_{:,j}))}{\text{avg}_j(\text{norm}(\mathbf{G}_{:,j}))} \mathbf{1}^\top, \\ \mathbf{G} &= g(\mathbf{h}; \Theta), \end{aligned} \quad (6)$$

where $\odot$ denotes element-wise multiplication, $\mathbf{M}$ denotes the rescaling matrix, norm computes the L_1 norm of the input vector, and $\mathbf{1} \in \mathbf{R}^r$ denotes the all-one vector. The equations in Eq. 3 and Eq. 6 are compiled as GaRA.

Table 1. Evaluation Results on Zero-shot Dataset Transfer (measured by accuracy for node classification tasks and AUC for graph classification tasks: %). “NULL” indicates unsupported tasks.

	NODE CLASSIFICATION				GRAPH CLASSIFICATION		LINK PREDICTION	
	CORA	WIKICS	REDDIT	INSTAGRAM	BACE	HIV	AMAZON PHOTO	BOOK HISTORY
VICUNA-7B	0.155	0.29	0.309	0.391	0.492	0.467	0.576	0.503
OFA	0.189	0.361	0.498	0.580	0.483	0.404	0.499	0.457
GRAPHGPT	0.126	-	-	-	-	-	-	-
LLAGA	0.156	0.601	0.499	0.397	NULL	NULL	0.659	0.602
TEA-GLM	0.202	0.449	0.491	0.479	0.467	0.498	0.675	0.584
GOFA	0.039	0.613	0.493	0.367	0.500	0.481	0.504	-
UNIYTE	0.216	0.635	0.512	0.603	0.534	0.501	0.681	0.515
GARA (OURS)	0.251	0.669	0.518	0.611	0.549	0.512	0.708	0.722

4.4. Model Training Strategy

Following the common practice in graph compression strategy (Chen et al., 2024a; Wang et al., 2025), we adopt instruction tuning during the model training. Specifically, let $\mathcal{D} = \{(\mathcal{G}^{(i)}, \mathcal{G}_s^{(i)}, t_{\mathcal{G}_s}^{(i)}, t_{\text{task}}^{(i)}, t_{\text{label}}^{(i)})\}_{i=1}^D$ be the training dataset, where $t_{\mathcal{G}_s}^{(i)}$ denotes the i -th subgraph node sequence, $t_{\text{task}}^{(i)}$ and $t_{\text{label}}^{(i)}$ denotes the corresponding task description and label, respectively. The model is required to maximize the log-likelihood function

$$\Theta_{\text{GaRA}}^* = \arg \max_{\Theta_{\text{GaRA}}} \sum_{i=1}^D \log P_{\Theta_{\text{GaRA}}}(t_{\text{label}}^{(i)} | I_i), \quad (7)$$

$$I_i = (\mathcal{G}^{(i)}, \mathcal{G}_s^{(i)}, t_{\mathcal{G}_s}^{(i)}, t_{\text{task}}^{(i)}),$$

where $P_{\Theta_{\text{GaRA}}}$ denotes the conditional probability distribution, Θ_{GaRA} denotes the set of trainable parameters during model fine-tuning. This target can be achieved by minimizing the cross-entropy loss

$$\mathcal{L} = - \sum_{i=1}^D \sum_{j=1}^{|t_{\text{label}}^{(i)}|} \log P_{\Theta_{\text{GaRA}}}(t_{\text{label},j}^{(i)} | t_{\text{label},<j}^{(i)}, I_i), \quad (8)$$

where $t_{\text{label},<j}^{(i)} = (t_{\text{label},k}^{(i)} | k < j)$ denotes all the tokens that before the j -th token in the output sequence.

5. Experiments

In this section, we provide empirical evaluation results of GaRA on real-world benchmarks. GaRA² is implemented with PyTorch (Paszke et al., 2019) and PyTorch Geometric (Fey & Lenssen, 2019). The detailed experimental settings are presented in Appendix B.

5.1. Effectiveness Study on Zero-shot Dataset Transfer

Experimental Setup. To evaluate the effectiveness of GaRA in generalizing to unseen datasets, we split train-

ing and inference sources into two non-overlapping sets of datasets. Specifically, GaRA and baselines are fine-tuned on five datasets: arXiv (Hu et al., 2020), BookChildren (Yan et al., 2023), AmazonComputer (Shchur et al., 2019), FB15K237 (Liu et al., 2023), and ChEMBL (Chen et al., 2024b), encompassing node classification, link prediction, and graph classification tasks. On arXiv, BookChildren, and AmazonComputer, both node classification and link prediction tasks are included. After fine-tuning, GaRA and baseline models are evaluated in a zero-shot setting on unseen datasets, including Cora (Wen & Fang, 2023), WikiCS (Mernyei & Cangea, 2022), Reddit (Li et al., 2024b), and Instagram (Li et al., 2024b) for node classification, (AmazonPhoto and BookHistory) (Yan et al., 2023) for link prediction, and (ChemHIV and ChemBACE) (Wu et al., 2018) for graph classification. Dataset statistics are summarized in Tab. A4.

Baseline models include the output-level injection method OFA (Liu et al., 2023), input-level injection methods (GraphGPT (Tang et al., 2024), LLaGA (Chen et al., 2024a), TEA-GLM (Wang et al., 2024), GOFA (Kong et al., 2024), and UniGTE (Wang et al., 2025)), and a pre-trained LLM Vicuna-7B (Zheng et al., 2023). GaRA is implemented with GCN (Kipf & Welling, 2017) and MaxPooling for whole-graph information extraction. In practice, the graph-aware weight generation is only applied to the 21-st layer. Please refer to Sec. 5.3 for more ablation details.

Performance. Tab. 1 shows the comparison results between GaRA and baseline models for zero-shot dataset transfer, where GaRA consistently achieves better performance across various datasets and task types. For node classification and link prediction tasks, methods that solely depend on the input-level injection can only encode subgraph information, where the whole-graph information is discarded. In contrast, GaRA further performs weight-level injection by explicitly incorporating whole-graph information, encoding both local features at the input level and higher-level features at the weight level. This design avoids potential information loss in the discarded whole graph, resulting in consistently

²Codes are available at <https://github.com/sunjss/GaRA>.

Table 2. Evaluation Results in Tackling Norm Amplification (measured by accuracy for node classification tasks and AUC for graph classification tasks: %). “Partial” denotes employing arXiv, BookChildren, AmazonComputer, and FB15K237 for tuning, where the first three datasets only involve node classification tasks. “Full” denotes employing arXiv, BookChildren, AmazonComputer, FB15K237, and ChEMBL for tuning, where the first three datasets involve both node classification and link prediction tasks. “NR.” denotes norm rescaling. “Rel. Imp.” denotes relative improvement of employing norm rescaling in GaRA compared to no explicit norm constraints.

TRAINING DATA	MODEL	LINK PREDICTION		NODE CLASSIFICATION			
		AMAZON PHOTO	BOOK HISTORY	CORA	REDDIT	WIKICS	INSTAGRAM
PARTIAL	GARA (w/o NR)	0.527	0.580	0.203	0.502	0.611	0.569
	GARA (w NR)	0.529	0.591	0.208	0.513	0.664	0.607
	REL. IMP.	0.37%	1.89%	2.46%	2.19%	8.67%	6.67%
FULL	GARA (w/o NR)	0.539	0.581	0.164	0.506	0.603	0.583
	GARA (w NR)	0.708	0.722	0.251	0.518	0.669	0.611
	REL. IMP.	31.35%	24.26%	53.04%	2.37%	10.94%	4.80%

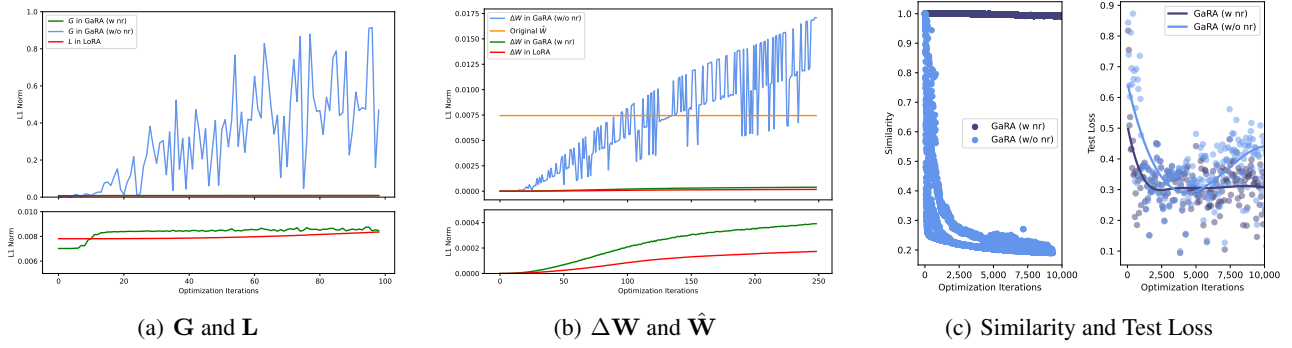


Figure 3. Effectiveness in Tackling Norm Amplification. “nr” denotes our norm rescaling strategy. (a) and (b) The lower figures are the zoom-in illustrations of the upper figures. (c) “Similarity” denotes the cosine similarity between the updated weight $\mathbf{W}$ and the original weight $\hat{\mathbf{W}}$. Results are averaged over the 50 randomly selected test samples, where the selection is refreshed every 50 training steps.

better performance of GaRA on both node and link tasks. Moreover, on graph classification tasks, where whole-graph information is directly provided to LLMs, GaRA also surpasses baseline models. We attribute this advantage to the explicit injection of whole-graph representations, whereas input-injection methods must implicitly infer global structure through pairwise node interactions, which can be less effective for capturing graph representations.

5.2. Effectiveness in Tackling Norm Amplification

To study the effectiveness of GaRA in tackling norm amplification, we start by presenting the norm values of the generated low-rank matrix with or without norm rescaling in Fig. 3(a) and 3(b). Then we dive into the potential influence of norm amplification. Based on the analysis in Section 4.3, norm amplification may lead to dilution of the original weight and model overfitting. Here, Fig. 3(c) presents the cosine similarity between the updated weight $\mathbf{W}$ and the original weight $\hat{\mathbf{W}}$ for weight dilution and the test loss for model overfitting. The results are from the value matrix projection in the 21-st layer of the LLM encoder Vicuna-7B.

As shown in Fig. 3(a), without norm rescaling, the fine-tuning process exhibits continuous amplification on the norm of the generated low-rank matrix. By employing the generated low-rank matrix to compute the weight update $\Delta \mathbf{W}$, either the norm or the direction of the updated weight $\mathbf{W}$ is dominated by the update part over the original weight $\hat{\mathbf{W}}$ (Fig. 3(b) and Fig. 3(c) left). As a result, the model without norm rescaling is prone to overfitting (Fig. 3(c) right), leading to degraded performance on transfer tasks. In contrast, GaRA with norm rescaling better preserves the knowledge in the original weights, while only partially adjusting the final weights through the generated updates.

Besides the analysis on model characteristics, Tab. 2 further presents the performance comparison with and without norm rescaling. It shows that weight generation without norm rescaling achieves inferior performance compared to generating with norm rescaling. When the amount of training data increases, norm rescaling empowers GaRA to gain enhanced transferability, while generating without norm rescaling gains limited improvements. Therefore, the performance gap between the two methods becomes larger.

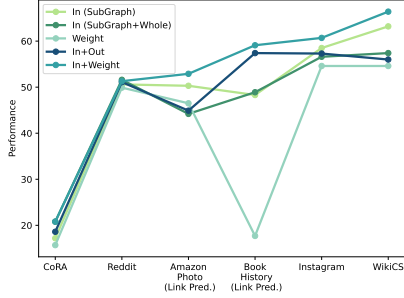


Figure 4. Comparison Among Injection Paradigms. “Sub-Graph” solely employs subgraph features as input. “Whole” further concatenates global features to the input sequences.

All these results indicate the negative impact of the norm amplification during optimization and the effectiveness of our norm-rescaling strategy in tackling this architectural optimization bias. Except for norm rescaling, there are other potential strategies for constraining norm amplification. Please refer to Appendix B.5 for more details.

5.3. Ablation Study

In this section, we conduct ablation studies on the architectural designs in GaRA, including comparing GaRA with other injection solutions, the position choice for weight-level graph information injection, and different generating inputs and targets. Model variants are fine-tuned on arXiv, BookChildren, AmazonComputer, and FB15K237, where arXiv, BookChildren, and AmazonComputer contain only node classification tasks.

5.3.1. IMPACT OF THE INJECTION PARADIGM

GaRA combines input-level and weight-level injection to provide graph structures for LM backbones. To isolate the contribution of our weight-level injection, we compare parameter-matching models under the same UniGTE framework with subgraph input injection, whole-graph input injection, weight injection, input-weight injection (GaRA), and input-output injection. Specifically, subgraph input injection is based on UniGTE, while whole-graph input injection directly concatenates whole-graph information learned by GNN at the input level. Weight injection gains graph information solely from the weight generator by masking the graph embedding in the LM input. Input-output injection applies a GNN to the whole graph to extract node-level representations and add them to the LLM encoder output. Results are presented in Fig. 4, following the “Partial” setup in Tab. 2. We can see that combining input and weight injection consistently achieves better performance than other variants, indicating that the two injection paradigms provide complementary benefits.

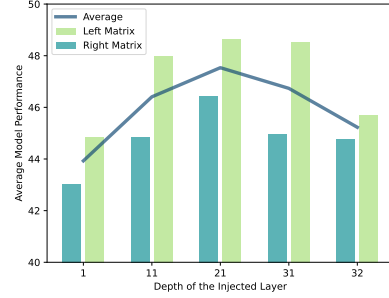


Figure 5. Impact of the Injection Position and Generation Target. “Left/Right” denotes that the generated low-rank matrix serves as an augmented left or right matrix.

5.3.2. IMPACT OF THE WEIGHT INJECTING POSITION

In practice, GaRA applies weight-level injection to only a single layer of the backbone LLM. To study the impact of different injection positions, we adopt Vicuna-7B as the backbone model, which consists of 32 layers. Injection positions are selected from {1, 11, 21, 31, 32}-th layer. Fig. 5 reports the average performance across the node classification and link prediction datasets from Tab. 1. We can see that injecting at the 21-st layer achieves the best average performance. As the injection position moves from shallow to intermediate layers, performance gradually improves and then degrades near the output layer. This is because injecting graph-aware features into deeper layers yields a more direct influence on the final predictions. However, the recovery trend does not extend to the final layer, as such late-stage perturbations bypass most of the backbone’s forward transformations and disrupt the final output representations.

5.3.3. IMPACT OF THE GENERATION TARGET

GaRA adopts low-rank fine-tuning to adapt pre-trained LLMs to graph tasks. During the generation process, either the left or right low-rank matrix can serve as the weight-level injection target. To investigate the impact of the two generation targets, we conduct an ablation study by employing the generated low-rank matrix $\mathbf{G}$ as an augmented left or right low-rank matrix. Following the same experimental setup in Sec 5.3.2, Fig. 5 compares the average performance with the two generation targets. Results show that generating the augmented left low-rank matrix consistently outperforms the augmented right matrix. We also note that either generating the left or the right matrix for layers at different depths follows the same performance trend in Sec. 5.3.2, further verifying our conclusion.

5.3.4. IMPACT OF THE GENERATION INPUT

GaRA employs the whole-graph representation to generate weight updates. To validate the benefit of whole-graph

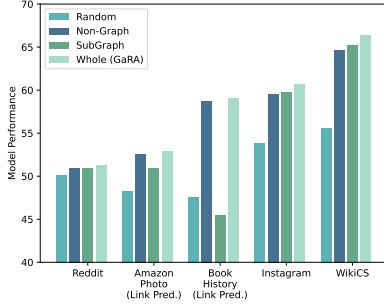


Figure 6. **Comparison Among Generation Inputs.** “Random”, “Non-Graph”, and “SubGraph” denote generating low-rank matrices from random vectors, embedded task description, and GNN-processed subgraph features, respectively.

Table 3. **Cost Analysis (s/sample).**

	UNIGTE	GARA	LLAGA	VICUNA-7B
TRAIN	1.28	1.46	0.83	-
INFERENCE	0.67	0.71	0.12	1.32

information, we compare different generation inputs, including random vectors, non-graph representations from the embedded task description, subgraph representations, and whole-graph representations (GaRA). We also compare GaRA with an MLP variant to isolate the contribution of graph-level representations. Please refer to Appendix B.4 for details. Results in Fig. 6 show that GaRA consistently achieves better performance than other variants. Specifically, random vectors cannot provide any informative features and result in the worst performance. Non-graph and subgraph representations achieve similar performance, where discarding whole-graph information leads to inferior performance. This stems from the fact that global features are complementary to the input-level local features, while adding more local features shows redundancy.

To further demonstrate such redundancy, we compute the cosine similarity between hidden representations obtained with pure LoRA versus the variant with subgraph representations and GaRA. 50 samples are randomly selected from the test sets. Averaged results in Fig. 7 show that generating with local information offers similar features to the original features, indicating redundancy. In contrast, generating with global information offers less similar features to the original ones, indicating complementarity. All these results demonstrate the importance of further complementing whole-graph information for the primary input injection.

5.4. Cost Analysis

GaRA depends on a GNN model to learn global features and a single-layer perceptron to generate the low-rank weight matrix. In practice, we employ GCN (Kipf & Welling, 2017) as the backbone to construct our GNN model, which re-

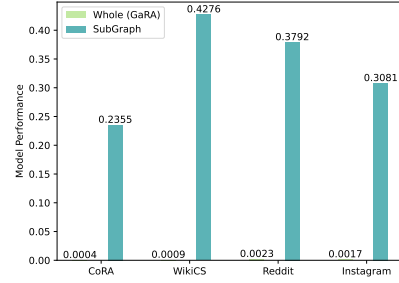


Figure 7. **Redundancy Analysis with Similarity Comparison.** Cosine similarity is computed between hidden representations obtained with the vanilla LoRA versus GaRA with “Whole” graph/“SubGraph” as generator inputs.

quires a computational complexity of $O(m) + O(n)$. Therefore, the computational bottleneck mainly comes from the LLM backbone instead of our external weight generation module. To empirically evaluate the efficiency of GaRA, we conduct a cost analysis during both the tuning and the inference processes. Specifically, we employ (arXiv, BookChildren, AmazonComputer, and FB15K237) as the training datasets, and (Cora, BookHistory, and AmazonPhoto) as the inference datasets, computing the average cost per sample. Results in Tab. 3 show that GaRA stays on par with other LLM-based models.

6. Conclusion

In this paper, we proposed a novel weight-level injection paradigm for adapting LMs to graph tasks. Unlike prior input- or output-level methods, which either overlook higher-level information from the whole-graph context or struggle to generalize across tasks, our paradigm enables explicit whole-graph injection by generating weight updates during fine-tuning and applying them to hidden-layer representations. In this way, weight-level injection complements subgraph information introduced at the input level and mitigates potential information loss. Building on this paradigm, we then proposed a graph-aware weight generation method, named GaRA. GaRA consists of a LoRA-style weight generation module and a norm-rescaling module. The weight generation module produces weight updates that inject whole-graph information without incurring a quadratic increase in computational cost. The norm-rescaling module addresses a direction-preserving norm amplification issue, which we identify and theoretically characterize as an architectural optimization bias in primary weight generation. Empirical results demonstrate that GaRA consistently outperforms existing baselines on zero-shot transfer tasks and effectively alleviates norm amplification, validating both the effectiveness and robustness of the proposed approach. For the limitation discussion, please refer to Appendix C.

Acknowledgement

This work was supported in part by the National Key R&D Program of China under Grant 2023YFC2508704, in part by the National Natural Science Foundation of China under grant number 62236008, in part by the Natural Science Foundation of Beijing under grant number L251082, and in part by Shandong Provincial Natural Science Foundation under project ZR2025ZD01. The authors would like to thank the anonymous reviewers for their helpful comments and suggestions that improved this manuscript.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Introducing Llama 3.1: Our most capable models to date. <https://ai.meta.com/blog/meta-llama-3-1/>, 2024.
- Bessadok, A., Mahjoub, M. A., and Rekik, I. Graph Neural Networks in Network Neuroscience. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(5):5833–5848, 2023. ISSN 1939-3539.
- Chen, R., Zhao, T., Jaiswal, A. K., Shah, N., and Wang, Z. LLaGA: Large Language and Graph Assistant. In *International Conference on Machine Learning*, pp. 7809–7823. PMLR, 2024a.
- Chen, Z., Mao, H., Liu, J., Song, Y., Li, B., Jin, W., Fatemi, B., Tsitsulin, A., Perozzi, B., Liu, H., and Tang, J. Text-space Graph Foundation Models: Comprehensive Benchmarks and New Insights. In *Advances in Neural Information Processing Systems*, volume 37, pp. 7464–7492, December 2024b.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.
- Fey, M. and Lenssen, J. E. Fast Graph Representation Learning with PyTorch Geometric. In *International Conference on Learning Representations Workshop on Graphs and Manifolds*, 2019.
- Hamilton, W. L., Ying, R., and Leskovec, J. Inductive representation learning on large graphs. In *International Conference on Neural Information Processing Systems*, pp. 1025–1035, Red Hook, USA, 2017. Curran Associates Inc.
- He, Y., Sui, Y., He, X., and Hooi, B. UniGraph: Learning a Unified Cross-Domain Foundation Model for Text-Attributed Graphs. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, KDD ’25, pp. 448–459, New York, NY, USA, 2025. Association for Computing Machinery. doi: 10.1145/3690624.3709277.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2021.
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *Advances in Neural Information Processing Systems*, volume 33, pp. 22118–22133. Curran Associates, Inc., 2020.
- Kipf, T. N. and Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*, Toulon, France, 2017.
- Kong, L., Feng, J., Liu, H., Huang, C., Huang, J., Chen, Y., and Zhang, M. GOfA: A Generative One-For-All Model for Joint Graph Language Modeling. In *The Thirteenth International Conference on Learning Representations*, October 2024.
- Li, G., Müller, M., Qian, G., Delgadillo, I. C., Abualshour, A., Thabet, A., and Ghanem, B. DeepGCNs: Making GCNs Go as Deep as CNNs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–1, 2021. ISSN 0162-8828, 2160-9292, 1939-3539. doi: 10/gk3x7j.
- Li, Y., Wang, P., Li, Z., Yu, J. X., and Li, J. ZeroG: Investigating Cross-dataset Zero-shot Transferability in Graphs. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1725–1735, New York, NY, USA, 2024a. Association for Computing Machinery.
- Li, Y., Wang, P., Zhu, X., Chen, A., Jiang, H., Cai, D., Chan, V. W., and Li, J. GLBench: A Comprehensive Benchmark for Graph with Large Language Models. *Advances in Neural Information Processing Systems*, 37: 42349–42368, 2024b.
- Lim, D., Hohne, F., Li, X., Huang, S. L., Gupta, V., Bhalerao, O., and Lim, S. N. Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods. In *Advances in Neural Information Processing Systems*, volume 34, pp. 20887–20902. Curran Associates, Inc., 2021.

- Liu, H., Feng, J., Kong, L., Liang, N., Tao, D., Chen, Y., and Zhang, M. One For All: Towards Training One Graph Model For All Classification Tasks. In *International Conference on Learning Representations*, 2023.
- lv, R., Zhang, Z., Zhang, K., Liu, Q., Gao, W., Liu, J., Yan, J., Yue, L., and Yao, F. GraphPrompter: Multi-Stage Adaptive Prompt Optimization for Graph In-Context Learning. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pp. 3917–3930. IEEE Computer Society, 2025. doi: 10.1109/ICDE65448.2025.00292.
- Ma, F., Xue, H., Zhou, Y., Wang, G., Rao, F., Yan, S., Zhang, Y., Wu, S., Shou, M. Z., and Sun, X. Visual perception by large language model’s weights. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, volume 37, pp. 28615–28635, Red Hook, NY, USA, 2024. Curran Associates Inc.
- Mernyei, P. and Cangea, C. Wiki-CS: A Wikipedia-Based Benchmark for Graph Neural Networks. In *International Conference on Machine Learning Workshop on Graph Representation Learning and Beyond*. arXiv, January 2022.
- Morris, C., Kriege, N. M., Bause, F., Kersting, K., Mutzel, P., and Neumann, M. TUDataset: A collection of benchmark datasets for learning with graphs. In *International Conference on Machine Learning Workshop on Graph Representation Learning and Beyond*, Virtual Only, 2020. arXiv.
- OpenAI, Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A. J., Welihinda, A., Hayes, A., Radford, A., Madry, A., Baker-Whitcomb, A., and Beutel, A. e. a. GPT-4o System Card, 2024.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: An imperative style, high-performance deep learning library. In *International Conference on Neural Information Processing Systems*, pp. 8026–8037, Red Hook, NY, USA, 2019. Curran Associates Inc.
- Reimers, N. and Gurevych, I. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019.
- Shchur, O., Mumme, M., Bojchevski, A., and Günnemann, S. Pitfalls of Graph Neural Network Evaluation. *arXiv:1811.05868 [cs, stat]*, 2019.
- Sun, J., Wang, S., Han, X., Xue, Z., and Huang, Q. All in a row: Compressed convolution networks for graphs. In *International Conference on Machine Learning*, volume 202, pp. 33061–33076, Honolulu, USA, 2023. PMLR.
- Sun, J., Yang, C., Ji, X., Huang, Q., and Wang, S. Towards Dynamic Message Passing on Graphs. In *Conference on Neural Information Processing Systems*, December 2024.
- Sun, J., Chang, W., Yang, C., Huang, Q., and Wang, S. Relieving the Over-Aggregating Effect in Graph Transformers. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Tang, J., Yang, Y., Wei, W., Shi, L., Su, L., Cheng, S., Yin, D., and Huang, C. GraphGPT: Graph Instruction Tuning for Large Language Models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 491–500, New York, NY, USA, 2024. Association for Computing Machinery.
- Van der Maaten, L. and Hinton, G. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.
- Wang, D., Zuo, Y., Li, F., and Wu, J. LLMs as zero-shot graph learners: Alignment of GNN representations with LLM token embeddings. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, volume 37, pp. 5950–5973, Red Hook, NY, USA, 2024. Curran Associates Inc.
- Wang, D., Zuo, Y., Lu, G., and Wu, J. UniGTE: Unified Graph-Text Encoding for Zero-Shot Generalization across Graph Tasks and Domains. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Wang, H., Feng, S., He, T., Tan, Z., Han, X., and Tsvetkov, Y. Can language models solve graph problems in natural language? In *International Conference on Neural Information Processing Systems*, pp. 30840–30861, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Wen, Z. and Fang, Y. Augmenting Low-Resource Text Classification with Graph-Grounded Pre-training and Prompting. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 506–516, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3539618.3591641.
- Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leswing, K., and Pande, V. MoleculeNet: A benchmark for molecular machine learning. *Chemical Science*, 9(2):513–530, 2018. ISSN 2041-6520. doi: 10.1039/c7sc02664a.

- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. How Powerful are Graph Neural Networks? In *International Conference on Learning Representations*, New Orleans, LA, USA, 2019.
- Yan, H., Li, C., Long, R., Yan, C., Zhao, J., Zhuang, W., Yin, J., Zhang, P., Han, W., Sun, H., Deng, W., Zhang, Q., Sun, L., Xie, X., and Wang, S. A comprehensive study on text-attributed graphs: Benchmarking and rethinking. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pp. 17238–17264, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., and Liu, T.-Y. Do Transformers Really Perform Bad for Graph Representation? In *Advances in Neural Information Processing Systems*, volume 34, pp. 28877–28888. Curran Associates, Inc., 2021.
- Zhang, L., Chen, M., Arnab, A., Xue, X., and Torr, P. H. S. Dynamic Graph Message Passing Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 5712–5730, 2022.
- Zhang, M. and Chen, Y. Link Prediction Based on Graph Neural Networks. In *International Conference on Neural Information Processing Systems*, pp. 5171–5181, Red Hook, NY, USA, 2018. Curran Associates Inc.
- Zhang, M., Sun, M., Wang, P., Fan, S., Mo, Y., Xu, X., Liu, H., Yang, C., and Shi, C. GraphTranslator: Aligning Graph Model to Large Language Model for Open-ended Tasks. In *The Web Conference 2024*, May 2024.
- Zhao, J., Zhu, Z., Galkin, M., Mostafa, H., Bronstein, M. M., and Tang, J. Fully-inductive Node Classification on Arbitrary Graphs. In *The Thirteenth International Conference on Learning Representations*, 2024.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.

Table A4. **Dataset Statistics.** “NC”, “LP”, “GC” denote node classification, link prediction, and graph classification, respectively.

TYPE	DATASET	TASK	#NODES	#EDGES	#CLASSES/TAKS	#GRAPHS
TRAIN	ARXIV	NC, LP	169,343	1,166,243	40	1
TRAIN	AMAZONCOMPUTERS	NC, LP	87,229	721,081	10	1
TRAIN	BOOKCHILDREN	NC, LP	76,875	1,554,578	24	1
TRAIN	FB15K237	LP	14,541	310,116	237	1
TRAIN	CHEMBL	GC	26	56	1,048	365,065
TEST	INSTAGRAM	NC	11,339	144,010	2	1
TEST	REDDIT	NC	33,434	198,438	2	1
TEST	WIKICS	NC	11,701	216,123	10	1
TEST	CORA	NC	25,120	91,140	70	1
TEST	BOOKHISTORY	LP	41,551	358,574	12	1
TEST	AMAZONPHOTO	LP	48,362	500,928	12	1
TEST	CHEMBACE	GC	34	74	1	1,513
TEST	CHEMHIV	GC	25,51	54,95	1	41,127

A. Proof

Theorem 4.1 (Direction-preserving Update). *Let $\Delta\mathbf{W} = \mathbf{f}(\mathbf{h}; \Theta)$. For a fixed input $\mathbf{h}$ and the Jacobian $J_{\mathbf{f}}(\Theta) = \partial \mathbf{f}(\mathbf{h}; \Theta) / \partial \Theta$, there exists a nonzero parameter direction $\delta\Theta$ such that under the update $\Theta \leftarrow \Theta + \epsilon \delta\Theta$, the induced weight update satisfies $\Delta\mathbf{W} \leftarrow (1 + \epsilon)\Delta\mathbf{W} + O(\epsilon^2)$.*

Proof. Based on the first-order Taylor expansion

$$\mathbf{f}(\mathbf{h}; \Theta + \epsilon \delta\Theta) = \mathbf{f}(\mathbf{h}; \Theta) + \epsilon J_{\mathbf{f}}(\Theta) \delta\Theta + O(\epsilon^2). \quad (\text{A9})$$

Our target is to achieve $\mathbf{f}(\mathbf{h}; \Theta + \epsilon \delta\Theta) = (1 + \epsilon)\mathbf{f}(\mathbf{h}; \Theta)$. Substituting Eq. A9 into the above equation yields

$$\mathbf{f}(\mathbf{h}; \Theta) + \epsilon J_{\mathbf{f}}(\Theta) \delta\Theta + O(\epsilon^2) = (1 + \epsilon)\mathbf{f}(\mathbf{h}; \Theta). \quad (\text{A10})$$

Ignoring higher-order terms, this equation is reduced to a linear function

$$J_{\mathbf{f}}(\Theta) \delta\Theta = \mathbf{f}(\mathbf{h}; \Theta). \quad (\text{A11})$$

In practice, $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^{p \times q}$ is typically implemented as a single-layer perceptron. Therefore, the number of parameters in $\mathbf{f}$ is no smaller than the output dimension $p \times q$. Under this implementation, the Jacobian $J_{\mathbf{f}}(\Theta)$ is full row-rank, there exists a solution $\delta\Theta$ for Eq. A11 given any $\mathbf{f}(\mathbf{h}; \Theta)$. The results also holds when $\mathbf{f}(\mathbf{h}; \Theta)$ is employed to generate the low-rank matrices for weight updates $\Delta\mathbf{W} = \mathbf{f}(\mathbf{h}; \Theta)\mathbf{R}^\top$ or $\Delta\mathbf{W} = \mathbf{L}\mathbf{f}(\mathbf{h}; \Theta)$, since left or right multiplication by low-rank matrices does not change the above derivation. This completes the proof. $\square$

Corollary 4.2 (Direction-preserving Update on LoRA-style Generation). *Let $\Delta\mathbf{W} = \mathbf{L}\mathbf{R}^\top + \mathbf{g}(\mathbf{h}; \Theta)\mathbf{R}^\top$. For a fixed input $\mathbf{h}$ and the Jacobian $J_{\mathbf{g}}(\Theta) = \partial \mathbf{g}(\mathbf{h}; \Theta) / \partial \Theta$, there exists a nonzero parameter direction $\delta\Theta$ such that under the update $\Theta \leftarrow \Theta + \epsilon \delta\Theta$, the induced weight update satisfies $\Delta\mathbf{W} \leftarrow (1 + \epsilon)\Delta\mathbf{W} + O(\epsilon)$.*

Proof. The result in Theorem 4.1 directly extends to the LoRA setting. Specifically, Eq. A11 can be written as

$$J_{\mathbf{g}}(\Theta) \delta\Theta = \mathbf{g}(\mathbf{h}; \Theta). \quad (\text{A12})$$

When $\mathbf{f}(\mathbf{h}; \Theta)$ is employed to generate the weight update via $\Delta\mathbf{W} = \mathbf{L}\mathbf{R}^\top + \mathbf{g}(\mathbf{h}; \Theta)\mathbf{R}^\top = \mathbf{f}(\mathbf{h}; \Theta)\mathbf{R}^\top$ or $\Delta\mathbf{W} = \mathbf{L}\mathbf{R}^\top + \mathbf{L}\mathbf{g}(\mathbf{h}; \Theta) = \mathbf{L}\mathbf{f}(\mathbf{h}; \Theta)$, we have $\mathbf{f}(\mathbf{h}; \Theta) = \mathbf{g}(\mathbf{h}; \Theta) + \mathbf{L}$ or $\mathbf{g}(\mathbf{h}; \Theta) + \mathbf{R}^\top$. Since $\mathbf{L}$ and $\mathbf{R}$ is independent of Θ , the Jacobian $J_{\mathbf{g}}(\Theta) = J_{\mathbf{f}}(\Theta)$, and thus remains full row rank. Therefore, Eq. A12 also admits a solution for any $\mathbf{g}(\mathbf{h}; \Theta)$, empowering the direction-preserving update property to hold in the low-rank scenario. $\square$

B. Details on Experimental Settings

B.1. Datasets

GaRA and baselines are fine-tuned on five datasets: arXiv (Hu et al., 2020), BookChildren (Yan et al., 2023), Amazon-Computer (Shchur et al., 2019), FB15K237 (Liu et al., 2023), and ChEMBL (Chen et al., 2024b), encompassing node classification, link prediction, and graph classification tasks. On arXiv, BookChildren, and AmazonComputer, both node classification and link prediction tasks are included. After fine-tuning, GaRA and baseline models are evaluated in a zero-shot setting on unseen datasets, including (Cora (Wen & Fang, 2023), WikiCS (Mernyei & Cangea, 2022), Reddit (Li et al., 2024b), and Instagram (Li et al., 2024b)) for node classification, (AmazonPhoto and BookHistory) (Yan et al., 2023) for link prediction, and (ChemHIV and ChemBACE) (Wu et al., 2018) for graph classification. The statistics of these datasets are summarized in Tab. A4. For data splitting, we follow standard splits for node classification and graph classification from their original papers. For link prediction, we randomly split the data into train/valid/test sets with a ratio of 8:1:1. During training, a subset of instances is sampled from each training dataset to stay under the same experimental setup as previous work (Wang et al., 2025). Specifically, 45,470 instances are sampled from arXiv, 21,888 from BookChildren, 31,378 from AmazonComputers for node classification tasks, 10,000 each from (arXiv, BookChildren, and AmazonComputers) and 29,440 from FB15K237 for link prediction tasks, and 74,242 from ChEMBL for graph classification. The task descriptions for each dataset are presented in Tab. A7.

B.2. Experimental Setup

GaRA follows UniGTE (Wang et al., 2025) to construct the input-injection framework and adds a graph-aware weight generation module to incorporate whole-graph information. Specifically, GaRA employs BERT (Devlin et al., 2019) as the pre-trained LM, and Vicuna-7B (Zheng et al., 2023) as the LLM backbone. During the model tuning process, the learning rate is set as $2e - 4$ for LoRA and the graph-aware weight generator, while the rest of the module follows the same setting as in UniGTE. All experiments are conducted on NVIDIA A100.

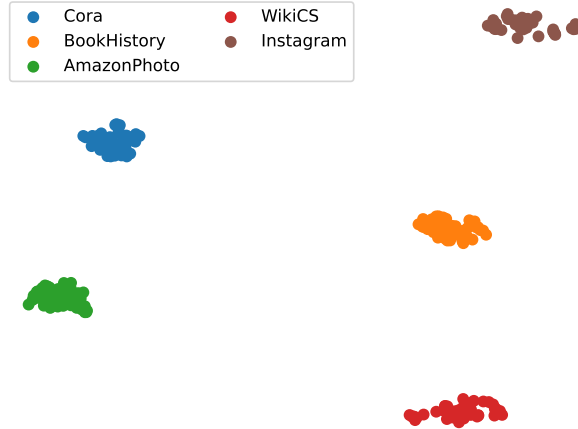


Figure A8. **Distribution of the Generated Weight Updates.** The t-SNE results of the generated weight updates from different datasets are compared. For each dataset, 20% of the node features are randomly masked to generate 50 perturbed views. Different colors indicate different datasets.

B.3. Graph-awareness of GaRA

GaRA encodes whole-graph information as weight updates for LLM adaptation. To verify whether the generated updates capture dataset-specific features, we conduct a graph-awareness analysis on the resulting low-rank matrices. Specifically, for each input graph, we randomly mask 20% of the node features to generate 50 perturbed views of the same graph. The generated low-rank matrices $\mathbf{G}$ from Cora, BookHistory, AmazonPhoto, Instagram, and WikiCS are then visualized using

Table A5. Evaluation Results with More Backbones (measured by accuracy for node classification tasks: %). “LP” denotes link prediction.

	VICUNA-7B					LLAMA3	
	IN	GCN	GAT	GIN	MLP	IN	GARA GCN
AMAZONPHOTO (LP)	50.3	52.9	52.4	52.5	51.8	51.2	56.6
BOOKHISTORY (LP)	48.3	59.1	58.9	58.4	48.6	44.1	51.3
INSTAGRAM	58.5	60.7	64.7	63.8	60.0	59.1	61.6
CORA	17.2	20.8	21.1	21.8	20.1	29.6	31.8
REDDIT	50.6	51.3	52.8	52.3	50.6	51.4	51.7
WIKICS	63.2	66.4	67.7	63.8	63.5	65.1	57.1

Table A6. Comparison with Different Strategies for Constraining Norm Amplification (measured by accuracy for node classification tasks: %). “LP” denotes link prediction. “No Constraint” denotes GaRA without norm rescaling.

	NO CONSTRAINT	NORM RESCALING (GARA)	NORMALI -ZATION	GATING	PENALTY	CLIPPING
AMAZONPHOTO (LP)	52.7	52.9	52.1	49.7	49.3	53.3
BOOKHISTORY (LP)	58.0	59.1	52.4	53.4	52.7	60.6
INSTAGRAM	56.9	60.7	60.1	59.2	60.4	61.3
CORA	20.3	20.8	11.6	18.7	10.7	18.8
REDDIT	50.2	51.3	50.6	51.0	50.7	50.2
WIKICS	61.1	66.4	18.7	53.7	57.5	63.3

t-SNE (Van der Maaten & Hinton, 2008). As shown in Fig. A8, different generating results $\mathbf{G}$ corresponding to the same dataset tend to cluster together, while results from different datasets form well-separated clusters. This observation suggests that GaRA encodes dataset-specific graph features, effectively empowering graph-aware adaptation of the LLM across diverse input graphs.

B.4. Evaluation with More Backbones

To evaluate the compatibility of GaRA with other backbones, GaRA is further implemented with MLP, GAT (Kipf & Welling, 2017), and GIN (Xu et al., 2019) for the GNN backbone. LLaMA3.1 8B-Instruct (Ila, 2024) is adopted as the LLM alternative. Experimental setup follows the “Partial” setting in Tab. 2. Results in Tab. A5 show that GaRA with different backbones consistently outperforms input-injection baselines, demonstrating its backbone compatibility. Notably, GaRA with different GNNs achieves better performance than the MLP variant. This indicates that the performance gains are from both graph representations and LoRA generation.

B.5. Strategies for Constraining Norm Amplification

Except for norm rescaling, there are other potential strategies for constraining norm amplification on the generated low-rank weight update $\mathbf{G}$. Specifically, we consider loss penalty by adding a loss term (Penalty), normalization by applying Frobenius normalization on $\mathbf{G}$ (Norm), element clipping by clipping elements in $\mathbf{G}$ that are larger than the largest element in the left low-rank matrix $\mathbf{L}$ from LoRA (Clipping), and learnable gating by multiplying a learnable scalar with $\mathbf{G}$ (Gating). Experimental setup follows the “Partial” setting in Tab. 2. Tab. A6 shows that GaRA with norm rescaling consistently outperforms Gating, Penalty, and Norm, while being comparable with Clipping, which also constrains the norm with the LoRA component $\mathbf{L}$. Therefore, both norm rescaling and clipping preserve useful norm information while preventing excessive norm amplification. These results suggest that effective constraints control the norm value and preserve meaningful scale information. Our norm rescaling follows these principles, while Clipping serves as a potential alternative. We will explore this direction in future work.

C. Limitation

In this paper, we introduce a weight-level injection paradigm that incorporates whole-graph information into LLM adaptation via weight generation. This paradigm is designed as a complementary strategy to input-level injection, which primarily relies on subgraph sequences and discards higher-level information from the original graphs. However, the proposed weight-level

paradigm has not yet been explored as a fully independent injection mechanism. In practice, the implemented model, GaRA, is built upon an input-level injection framework and leverages both input- and weight-level information. How to enable LLMs to perform graph tasks using only the weight-level injection paradigm, without relying on input-level prompts, remains an open problem and an interesting direction for future work.

Table A7. **Dataset Description.** “NC”, “LP”, “GC” denotes node classification, link prediction, and graph classification, respectively.

Datasets	Task Type	Description
arXiv, Cora	NC	Given a representation of a paper with the following information: Title: {title}, Abstract: {abstract}. Question: Which arXiv CS sub-category does this paper belong to? Please directly give the most likely answer from the following sub-categories: {candidate_labels}.
	LP	Given the representation of two papers: Title: First Paper: {title}, Second Paper: {title}. Question: Do these two papers have citation relationships? Please choose the most likely answer from: "Yes, they have citation relationships" or "No, they do not have citation relationships".
BookHistory, BookChildren	NC	Given a representation of a book with the following information: Name: {title}, Content: {abstract}. Question: Which category does this book belong to? Please directly give the most likely answer from the following categories: {candidate_labels}.
	LP	Given the representation of two books: Title: First Book: {title}, Second Book: {title}. Question: Do these two books have co-purchased or co-viewed relationships? Choose from: "Yes, they have co-purchased or co-viewed relationships" or "No, they do not have co-purchased or co-viewed relationships".
AmazonComputers, AmazonPhoto	NC	Given a representation of a book with the following information: Name: {title}, Content: {abstract}. Question: Which category does this book belong to? Please directly give the most likely answer from the following categories: {candidate_labels}.
	LP	Given the representation of two electronic products: Title: First Product: {comment}, Second Product: {comment}. Question: Do these two products have co-purchased or co-viewed relationships? Choose from: "Yes, they have co-purchased or co-viewed relationships" or "No, they do not have co-purchased or co-viewed relationships".
WikiCS	NC	Given a representation of a Wikipedia page with the following information: Name: {name}, Content: {content}. Question: Which category does this Wikipedia page belong to? Please directly give the most likely answer from the following categories: {candidate_labels}.
Reddit	NC	Given a representation of a user with the following information: Previous posts: {posts}. Question: Which category does this user belong to? Please directly give the most likely answer from the following categories: {candidate_labels}.
Instagram	NC	Given a representation of a user with the following information: Personal introduction: {introduction}. Question: Which category does this user belong to? Please directly give the most likely answer from the following categories: {candidate_labels}.
FB15K237	LP	Given the representation of two entities: First entity: Name: {name}, Description: {description}. Second entity: Name: {name}, Description: {description}. Question: Which category should the relation between these two entities be classified as? Please directly give the most likely answer from the following categories: {candidate_labels}.
ChEMBL, ChemHIV, ChemBACE	GC	Given a representation of a molecule with the following information: SMILES: {smiles}. Question: {task} Please answer: "Yes, this molecule is effective to this assay" or "No, this molecule is not effective to this assay".
	NC	Determine this node's most likely category within the network's classification schema
	LP	Determine whether there is a specific relationship between these two nodes.
	GC	Determine whether the molecule possesses specific physicochemical or bioactivity properties.